

MAlang Lexical Analyzer

Manual Scanner vs JFlex Scanner - Output Comparison

CS4031-Compiler Construction | Assignment 01 | Spring 2026

Moiz Ansari (23i-0523) | Abdullah Siddiqui (23-0617)

1. Overview

Both scanners were run on the same five test files (test1.malang through test5.malang). The outputs were compared token by token, error by error, and symbol-table entry by entry. This document records every observed difference, explains the root cause, and concludes with an overall assessment of the two implementations.

2. Token Count Summary

Test File	JFlex Tokens	Manual Tokens	Match?
test1.malang	71	71	✓ (same count)
test2.malang	102	102	✓ (same count)
test3.malang	30	30	✓ (same count)
test4.malang	45	~36 valid	≈ (error handling differs)
test5.malang	26	26	✓ (same count)

Note: Manual scanner does not explicitly print a total token count; count is inferred from symbol table total frequency.

3. Detailed Differences

3.1 Whitespace Token Type Name

The most consistent difference across all five test files is the token type name used for space characters.

Aspect	JFlex Output	Manual Scanner Output
Token type for ' '	SPACE	WHITE_SPACE
Example	<SPACE, " ", Line: 4, Col: 6>	<WHITE_SPACE, " ", Line: 4, Col: 7>

Aspect	JFlex Output	Manual Scanner Output
Affected tests	All five	All five

Reason: JFlex uses a dedicated SPACE rule and returns TokenType.SPACE. The manual scanner's returnCompatibleTokenName() maps state 16 to TokenType.WHITE_SPACE. Both are correct representations; they simply use different enum constants. The fix is to either rename one enum value or align both to use the same constant.

3.2 Column Number Offset (Off-by-One)

The manual scanner's column numbers are consistently offset by +1 compared to JFlex for the first token on each new line.

Token	JFlex Col	Manual Col	Difference
Count (line 4)	Col: 1	Col: 2	+1
Variable_name (line 5)	Col: 1	Col: 2	+1
42 (line 9)	Col: 1	Col: 2	+1
3.14 (line 12)	Col: 1	Col: 2	+1
((line 16)	Col: 1	Col: 2	+1

Reason: The manual scanner uses a boolean flag *fc* (first character) that delays the column increment by one step after a newline. This means the column counter is at 1 when the first character is read, but the column is only advanced *after* the first character, effectively reporting col 2 for what should be col 1. The fix is to initialize *col*=1 at line start and increment before using the value.

3.3 Newline Token Line Number Attribution

Newline tokens are attributed to different line numbers in the two scanners.

Token	JFlex	Manual Scanner
First \n (end of line 1)	<NEWLINE, "\n", Line: 1, Col: 1>	<NEWLINE, "\n", Line: 2, Col: 1>
Second \n (end of line 2)	<NEWLINE, "\n", Line: 2, Col: 1>	<NEWLINE, "\n", Line: 3, Col: 1>

Reason: JFlex reports the newline at the line it occupies (i.e., line 1). The manual scanner calls *updateLineAndCols()* *before* emitting the token, which increments the line counter

upon seeing `\n`, so the newline token is reported as being on the already-incremented next line. This is a side-effect of the order of operations in `updateLineAndCols()`.

3.4 Single-Line Comment Lexeme (Trailing Space)

JFlex includes a trailing space at the end of single-line comment lexemes; the manual scanner strips it.

Scanner	Lexeme of first comment
JFlex	"## test1.malang - All valid implemented tokens " (trailing space included)
Manual	"## test1.malang - All valid implemented tokens" (trailing space excluded)

Reason: The JFlex regex `##[^\n]*` greedily matches everything up to (but not including) the newline, including any trailing spaces. The manual scanner emits the comment token with `printToken(input, s, e-1, ...)`—the `e-1` trims the last character. For lines with a trailing space before `\n`, JFlex keeps it while the manual scanner does not. Functionally, this does not affect correctness.

3.5 Error Reporting Granularity (Test 4)

This is the most significant behavioral difference. For invalid lowercase identifiers like `count` and `myVariable`, the two scanners produce very different error counts.

Input	JFlex Errors	Manual Scanner Errors
count (line 4)	1 error → "Violation of Rule ... c" at col 1	5 errors → c (col 2), o (col 3), u (col 4), n (col 5), [space] (col 6)
myVariable (line 6)	1 error → "Violation of Rule ... m" at col 1	2 errors → m (col 2), y (col 3) - stops at 'Variable' which is a valid identifier

Input	JFlex Errors	Manual Scanner Errors
1.2345678 (line 12)	1 error (float too long)	1 error (float too long)
3. (line 13)	1 error (Unrecognized character '.')	1 error (Violation of Rule '.')
@.\$~ (line 16)	3 errors (one per symbol)	3 errors (one per symbol)
TOTAL	7 errors	12 errors

Reason: JFlex uses the regex `[a-z][a-z0-9_]*` as a single pattern (`Invalid_ID_Start`) and matches the entire lowercase word in one pass, reporting one error. The manual scanner processes one character at a time; since 'c', 'o', 'u', 'n', 't' are all lowercase and the DFA has no valid transition from state 1 on lowercase letters (other than t, f for keyword starts), each character individually triggers a "Violation of Rule" error before the scanner resets to state 1.

3.6 Missing Whitespace Tokens in Manual Scanner (Test 4)

Because the manual scanner fires individual errors for each character of an invalid word, it also consumes the space between the word and the next token as part of the error recovery loop, so that space never becomes a `WHITE_SPACE` token. JFlex, which skips the whole word in one step, correctly emits subsequent whitespace tokens.

4. Where Both Scanners Agree

Area	Result
Valid token recognition (tests 1-3, 5)	All tokens recognized with correct type and lexeme.
Symbol table contents	Identical entries: name, type, first line, frequency.
Total symbol table frequency (tests 1-3, 5)	71, 102, 30, 26 respectively.
Float literal constraint	Both detect 1.2345678 (max 6 decimals) as an error.
Invalid characters	Both report 3 errors with matching reasons for @, \$, ~.

Area	Result
Bracket / punctuator tokens	100% matching token types and lexemes.
Boolean literals	true/false correctly tokenized in all tests.
Signed integer/float recognition	+100, -567, -3.14, 1.5e10, 2.0E-3 all match.
31-character identifier limit	Maximum_length_identifier_12345 accepted by both.
TAB token (test 5)	Both emit <TAB, "\t"> at correct position.
Single-line comments	Both recognize ## and produce SINGLE_LINE_COMMENT.

5. Differences Summary

#	Difference	Impact	Fixable?
1	Space token: SPACE vs WHITE_SPACE	Minor (naming only)	Yes (align enum value)
2	Column offset +1 on line starts	Minor (position tracking)	Yes (fix fc flag logic)
3	Newline reported on next line number	Minor (cosmetic)	Yes (reorder updateLineAndCols)
4	Comment lexeme: trailing space included/excluded	Negligible	Yes (trim in regex or keep)
5	Error count: 7 (JFlex) vs 12 (Manual) on test4	Moderate (UX of error reporting)	Yes (match word-level recovery)
6	Missing whitespace tokens after error recovery	Negligible (tied to fix #5)	Yes (tied to fix #5)

6. Conclusion

Both scanners successfully tokenize all valid MAlang source code and produce equivalent symbol tables. The token types, lexemes, and frequencies match on every valid test file.

The six differences identified are all minor implementation details rather than fundamental correctness issues—no valid token is misclassified or missed by either scanner.

The most meaningful behavioral gap is in error recovery (Difference #5): JFlex performs word-level recovery using its `Invalid_ID_Start` pattern, cleanly reporting one error per invalid identifier. The manual scanner performs character-level recovery, reporting one error per invalid character. The JFlex approach produces cleaner, more user-friendly error messages and is closer to how real-world scanners behave.

Differences #2 and #3 (column/line offset) are purely cosmetic tracking bugs in the manual scanner that do not affect token recognition correctness. All other differences (#1, #4, #6) are negligible and do not affect the usability of either scanner for subsequent compiler phases.